Learning-Based Adaptive Transmission Control in Hybrid CoAP-MQTT IoT Systems

Ali Ahammad
Universidade Do Algarve
Faro, Portugal
aahammad@ualg.pt

Noelia Correia
NOVA LINCS
Faculdade de Ciências e Tecnologia
Universidade do Algarve
Faro, Portugal
ncorreia@ualg.pt

***Abstract*— Hybrid architectures that combine CoAP and MQTT work well for collecting IoT telemetry. However, it is still difficult to balance energy efficiency, timeliness, and reliability in dynamic environments. This article models transmission adaptation as a Markov Decision Process (MDP) and proposes a safety-constrained Double Deep Q-Network (DDQN) to optimize the system performance. The agent monitors contextual signals including battery level, delay/loss and backlog conditions, and data criticality to adapt MQTT Quality of Service (QoS) and reporting intervals. To improve operational safety under a scalar reward, the framework combines hard action masking, criticality-conditioned Q-value gating, and non-bypassable penalties that reduce unsafe outcomes. In a realistic simulator, the learned policy improves on-time delivery from 56% to 66-68% while reducing per-episode energy consumption by 10-12% compared with static baselines, suggesting longer device lifetime. Finally, a Dockerized CoAP–MQTT implementation with MQTT-to-InfluxDB logging is provided to support reproducible end-to-end experimentation and visualization.**

Keywords*— *IoT, CoAP; MQTT, QoS adaptation, reporting interval control, safe reinforcement learning, DDQN.

I. Introduction

The rapid growth of the Internet of Things (IoT) across industrial and urban sectors is driving the need for communication frameworks that are both resilient and resource-efficient. IoT devices usually have limited processing power and finite battery life, often operating over unstable wireless connections. Because of this, protocols need to be highly adaptive to maintain reliability without exhausting device resources [1], [2].

At present, the dominant standards to transfer data for IoT are the Constrained Application Protocol (CoAP) and Message Queuing Telemetry Transport (MQTT). CoAP is a lightweight request-response protocol that uses UDP and is designed for resource-constrained devices [3]. In comparison, MQTT uses TCP and a broker based architecture to provide robust publish/subscribe capabilities [4], [5]. CoAP offers CON/NON reliability levels, while MQTT has multiple levels of Quality of Service (QoS), but reliability or higher QoS comes with extra cost in terms of system resources. Many modern deployments use a hybrid CoAP-MQTT pipeline where CoAP is used for communication with sensing devices. On the other hand, MQTT is used as backend data distribution system. A critical limitation of such hybrid systems is that traditional deployments rely on static configurations, using fixed reporting intervals and constant QoS levels. However, IoT environments are usually dynamic. Fluctuations in battery levels, network latency, and the data urgency can lead to a mismatch that static settings can't fix. For example, strict rules might lead to unnecessary energy use when things are steady, or they might result in losing critical data during network congestion [6]. This underscores the need for an intelligent, adaptive control mechanism [7]. This article suggests that making sure important events are perceived is often more important than just delivering data packets accurately. Therefore, it recommends using a method that can adjust based on the situation when sending information. This leads to the main research question:

"Can a reinforcement learning agent dynamically adapt MQTT QoS and reporting intervals to extend system lifetime while maintaining superior reliability and timeliness compared to static baselines?"

This paper contributes: (1) a Markov Decision Process (MDP) framework for hybrid telemetry adaptation; (2) a safety-constrained DDQN variant with action masking and penalty-based gating; and (3) a containerized end-to-end implementation for reproducible performance analysis.

A. Hybrid CoAP–MQTT Architecture

IoT protocols need to balance low power consumption with network instability. While enterprise protocols like Advanced Message Queuing Protocol (AMQP) or Hypertext Transfer Protocol (HTTP) are often too resource-intensive, the combination between CoAP and MQTT provides a viable solution. In a typical hybrid setup, sensors act as CoAP servers or clients depending on the intended interaction. A gateway bridge translates these messages into MQTT publications to share with backend [8].

The CoAP-MQTT bridge can act as a practical control point besides translating messages between CoAP and MQTT. Every message passes through this bridge, so it is an ideal spot to manage how the data should flow. In this article, the bridge is set up to monitor the status of the devices and their connections. This information is sent to a separate controller that makes decisions. The controller looks at the data and chooses the best settings, like how often to send updates or how reliable the connection needs to be. The bridge then applies these settings in real time [8].

B. Protocol Interworking and Adaptive Sampling

A lot of research has focused on protocol translation, like using CoAP-to-HTTP gateways [9]. However, these studies [10], [11] mostly focus on the problems with connections rather than adapting settings for an efficient use of resources. Adaptive sampling with Deep Q-Networks (DQN) can reduce transmissions by considering energy cost, but it typically does not address constraint-aware delivery including QoS and rate control, across a CoAP–MQTT multi-protocol pipeline.

C. Reinforcement Learning for Network Control

Deep Reinforcement Learning (DRL) has become an effective tool for adjusting parameters [12], [13]. Recent studies, such as the iCoCoA framework, show that RL can optimize CoAP-specific parameters like timeout and backoff to mitigate congestion [14]. Also, research into MQTT QoS trade-offs highlights that using QoS levels 1 and 2 makes connections more reliable, but they also slow down data transfer and use more energy [15], [16]. Even with these insights, most literatures often treat these optimizations separately, or in a simplistic way.

While prior work covers gateway connectivity, sampling logic, and single-protocol tuning, there is a lack of research regarding multi-objective trade-offs via scalarization under safety-constrained RL controllers for hybrid pipelines. Specifically, there is limited evidence of a system that jointly optimizes MQTT QoS and reporting intervals by synthesizing context signals such as battery health, message backlog, network delay, and data criticality. This article addresses this gap by implementing a DDQN-based controller designed to balance the "escalate vs. Conserve" trade-off within a hybrid CoAP-MQTT environment.

II. Problem Formulation and Safety Constraints

A. Adaptive Control Problem

This article studies a hybrid CoAP–MQTT telemetry pipeline where the system must balance energy, timeliness, and reliability under non-stationary conditions. CoAP is used at the edge for low overhead. MQTT is used in the core for brokered delivery and QoS. The key problem is that fixed QoS and fixed reporting intervals do not behave well when the signal dynamics, backlog pressure, link delay, and battery level change over time.

The control goal is to learn a context-aware policy that conserves resources during calm periods and selectively escalates transmission service only when urgency/risk increases, such as with fast signal changes, anomalies, or degraded link conditions. The policy adapts MQTT QoS and reporting interval, and it may also adjust payload fidelity and a "critical" mode. The MDP elements and the safety mechanisms that constrain the policy are given next.

B. Critical Events and Class Imbalance

The system uses the idea of a critical operating window. A step is "critical" when the pipeline should prioritize reliability and timeliness over energy saving. In this article, criticality is detected from a small set of discrete indicators: fast signal change, high backlog, high loss, high delay, and an anomaly flag. Hard triggers mark clear emergencies such as, an anomaly flag or any indicator reaching its highest-severity value.

In the collected training data, critical states are a minority (about 22% critical vs. 78% non-critical). This is important because a learner might concentrate too much on calm states and fail during rare but important events. To tackle this issue, the model includes an auxiliary head that predicts a criticality probability, $p_{\text{crit}}(s)$, and it is trained with focal loss which reduces the weights of easy (non-critical) majority samples and places greater emphasis on rare or misclassified critical samples.

C. MDP Definition

The interaction is modelled as an episodic MDP [17]:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma) \quad (1)$$

At each step t, the agent observes the current state $s_t \in \mathcal{S}$, selects action $a_t \in \mathcal{A}$, receives a scalar reward $r_t = \mathcal{R}(s_t, a_t, s_{t+1})$, and the environment transitions to s_{t+1} according to transition probability $\mathcal{P}(s_{t+1}|s_t, a_t)$. The objective is to maximize the discounted return, $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$, where k is the nonnegative integer time offset (steps ahead), and with discount factor $\gamma \in [0,1]$.

State: The state is deliberately small and discrete. It contains urgency, timeliness, and resource context:

$$s_t = (\Delta T, bufT, B, L, D, I_{\text{cur}}, Q_{\text{cur}}, F_{\text{cur}}, anom) \quad (2)$$

In (2), ΔT is the rate of change of temperature data, $bufT$ is the backlog level, B is the battery level, L is the packet loss, D is the delay, the currently configured reporting interval is presented by I_{cur}, Q_{cur} is for current MQTT QoS level, F_{cur} is the current fidelity level, all these using $\{0,1,2\}$ bins (low/medium/high). The anomaly flag $anom$ is binary. These are one-hot encoded for the network input.

Action: The action controls the transfer settings:

$$a_t = (q, \tau, f, c) \quad (3)$$

In (3), $q \in \{0,1,2\}$ is the MQTT QoS, $\tau \in \{5,30,60\}$ seconds is the reporting interval, $f \in \{0,1,2\}$ is the payload fidelity, and $c \in \{0,1\}$ is a critical flag. This yields 54 discrete actions.

Reward: A shaped scalar reward is used to encourage information delivery, timeliness, reliability, and correct critical handling, while penalizing energy use and congestion. In short form:

$$r = U_t - C_t + R_{\text{idle}} - safety_pen + \Omega, \quad (4)$$

In (4), U_t aggregates positive utility terms for information delivery, reliable delivery and correct critical behavior, C_t aggregates costs for energy use, network stress and congestion backlog growth, $R_{\text{idle}} \geq 0$ is a small bonus for conservative operation during non-critical periods, $safety_pen \geq 0$ is the safety penalty which penalizes under-reaction during critical periods such as, selecting too-low QoS service when criticality is high, and $\Omega \leq 0$ is a non-bypassable penalty for catastrophic outcomes such as buffer overflow or missed critical delivery. In the implementation, the positive utility term U_t is formed from weighted subterms for information delivery, timeliness, reliability, and correct critical handling, using fixed coefficients $(w_{\text{I}}, w_{\text{T}}, w_{\text{R}}, w_{\text{C}}) = (0.30, 0.20, 0.28, 0.30)$ in all experiments. The remaining terms in (4), namely C_{t}, R_{idle}, $safety_pen$, and Ω, are then added exactly as indicated by the reward definition. After the full scalar reward is computed, it is scaled by a constant factor and clipped to $[-1,1]$ for numerical stability. This pragmatic scalarization was kept fixed across all experiments, but the Key Performance Indicator (KPIs) are reported separately, and an ablation study is included to isolate the safety-control contribution.

D. PASA-Inspired Rationale

The controller follows the Priority-based Adaptive Scheduling Algorithm (PASA) [18] principle in a simplified way: escalate when priority is high, conserve when priority is low. Here, priority is not computed by a scheduling algorithm. Instead, priority is represented by the state bins ΔT, $bufT$, B, L, D, and $anom$. High priority related to states are classified as "critical" by the hard triggers ($anom = 1$ or $bufT$ =2 or $L = 2$ or $D = 2$); a state is also considered as critical when a summed risk score over the binned indicators (ΔT, $bufT$, L, D), the anomaly flag, and a battery-risk term (0/1/2 for high/medium/low battery), reaches a fixed cutoff. All other state cases are treated as low priority. Actions with higher QoS, shorter intervals, higher fidelity, and $c = 1$ represent escalation. Actions with low QoS, long intervals, low fidelity, and $c = 0$ represent conservation. The reward and the safety constraints make escalation attractive in high-priority states.

E. Safety Constraints

A known weakness of scalar rewards is term cancellation: a positive utility can hide a serious safety cost. This article reduces that risk using a three-layer safety mechanism that acts before and after reward shaping [19]:

- Hard action masks remove clearly unsafe actions by forbidding very slow reporting under high ΔT /backlog/anomaly and disallowing QoS0 under severe loss/delay in a given state (rule-based feasibility).
- Soft criticality gating uses a learned criticality probability, $p_{\text{crit}}(s)$ and two thresholds to bias the action set toward $c = 1$ in critical states and toward $c = 0$ in calm states (with hysteresis to avoid rapid switching).

Non-bypassable penalty, Ω make catastrophic failures always dominate the return

III. System Implementation

A. System Architecture

The system is a containerized hybrid CoAP-MQTT pipeline with a closed-loop RL controller. A CoAP sensor digital twin generates telemetry and exposes it as a CoAP resource. A CoAP-MQTT bridge polls the CoAP resource and publishes telemetry to a local MQTT broker. The DDQN agent subscribes to telemetry topics, reconstructs the discretized state, selects a control action, and publishes it to an MQTT control topic. The bridge applies q when publishing to MQTT, while the sensor applies τ for sampling/reporting; f controls payload detail and c activates urgent mode. All telemetry and control actions are logged via an MQTT → InfluxDB service for offline analysis and Grafana visualization.

As shown in Fig.1, the design constitutes two logical pathways:

- Data Plane: This plane outlines how sensor data moves from devices to backend services. The path can be presented like: Sensor → CoAP server → CoAP—MQTT Telemetry Bridge → Local MQTT Broker → MQTT — InfluxDB Bridge → InfluxDB + Grafana.
- Control Plane: This plane shows how commands go back from the RL agent to the sensors. The path can be presented like this: MQTT → DDQN (RL) Controller → CoAP—MQTT Broker → Server (MQTT Control subscriber).

At each decision step, the agent outputs an action with four fields:

- MQTT QoS: The bridge uses the selected QoS when publishing telemetry to the broker

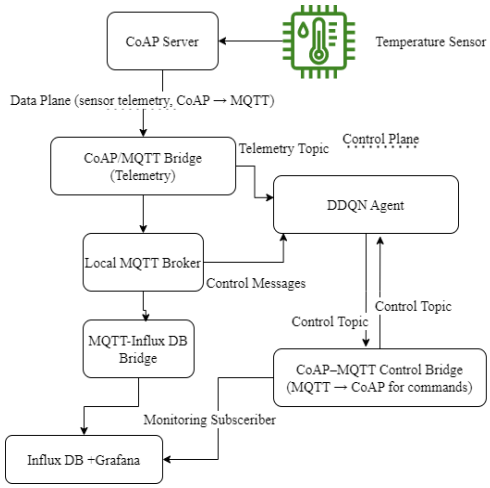

Fig. 1 Hybrid CoAP-MQTT system implementation.

- Reporting Interval: The CoAP server updates its sampling/reporting timer.
- Payload Fidelity: The bridge adjusts payload size by publishing fewer fields or using coarser values before forwarding to the broker.
- Critical Flag: It marks urgent states and triggers a short, reliable profile via the selected action.

Security (DTLS for CoAP and TLS for MQTT) is disabled in this study to avoid mixing cryptographic overhead with the core evaluation of energy-timeliness-reliability trade-offs.

B. State encoding and feature pipeline

At each step, raw measurements (signal change, backlog level, battery, loss, delay, anomaly) are converted into the nine discrete indicators in s_t. Binning keeps the policy compact and closer to what constrained devices can support. One-hot encoding avoids imposing false numeric distances between bins. The same state pipeline is used in simulation and in the containerized system to keep training and deployment consistent.

C. Action realization

Each chosen action $a_t = (q, \tau, f, c)$ is sent as a small control message through MQTT. The bridge uses q when publishing telemetry to the broker. The sensor uses τ to set its reporting period. Fidelity f controls how much detail is placed in the payload.

The flag c activates a short urgent mode such as treating the next transmissions as critical. The action is also logged so that learning decisions can be audited and compared.

IV. Learning Algorithm and Training Protocol

A. DDQN with Criticality-Aware Safety

The policy is learned using a DDQN that estimates $Q(s, a)$ for all discrete actions. Double-DQN target selection is used to reduce over-estimation. Training uses:

- Prioritized Experience Replay (PER) so rare and high-error transitions (often critical windows) are sampled more frequently [20].
- n-step returns to propagate delayed effects of congestion and energy costs more efficiently.
- Huber TD loss for stable value learning under outliers [21].
- An auxiliary criticality head that predicts $p_{\text{crit}}(s)$ and is trained with focal loss to address class imbalance [22].

During action selection, Q-values are filtered by hard masks and then adjusted by soft gating tied to $p_{\text{crit}}(s)$. This makes the action choice safety-constrained and criticality-aware, which makes action selection criticality-aware under explicit safety constraint.

B. Offline Training and Online Feasibility

Most learning and evaluation are performed in a realistic simulator that models signal dynamics, backlog evolution, loss/delay windows, and battery drain. This supports controlled experiments and ablations. The containerized CoAP-MQTT system is used to show that the same state/action interface can run end-to-end with real message exchange and logging. The article does not claim stable long-horizon online optimization results; instead, it uses the testbed to demonstrate deployability and reproducibility of the proposed control loop.

C. Ablation

Because the reward is scalarized and uses hand-chosen weights, this article includes an ablation [23] that removes safety modules (hard action masking and criticality-conditioned gating) to show two points clearly:

1. Safety layers reduce unsafe behaviour even when scalar reward learning can, in principle, trade off safety-critical terms against other utilities, and
2. Performance improvements are not only due to reward tuning, since the reward design is identical in both settings and only the safety layer is removed.

V. Experimental Setup

A. Simulator and Workload

A realistic simulator models a hybrid CoAP-MQTT telemetry pipeline with a constrained sensor, queue/backlog dynamics, and time-varying link quality. The temperature signal follows a day-night pattern with noise and occasional spikes to create short critical periods. Signal change is converted into bins (low/medium/high), and an anomaly flag is raised when change is unusually large.

B. State, Actions, and Safety

The controller observes a compact, binned state built from battery level, backlog pressure, delay/loss conditions, signal change, and anomaly/critical cues. Actions jointly select MQTT QoS and reporting interval (and optionally payload fidelity and a short critical mode). A three-layer safety design is used: hard action masks (forbidden actions removed), criticality-conditioned Q-value gating (bias toward escalate vs conserve), and non-bypassable penalties for catastrophic outcomes.

C. Training Protocol

Offline training uses 700 episodes, each with up to 250 steps. A step corresponds to one control interval $\tau \in \{5,30,60\}$ s. Fixed seeds are used for repeatable comparisons across configurations and baselines.

D. Baselines

The learned policy is compared against simple non-learning strategies:

- Static-frugal (low QoS, long interval),
- Static-aggressive (high QoS, short interval), and
- A PASA-style heuristic that escalates only when change/anomaly/backlog is high.

VI. Results

A. Energy Efficiency

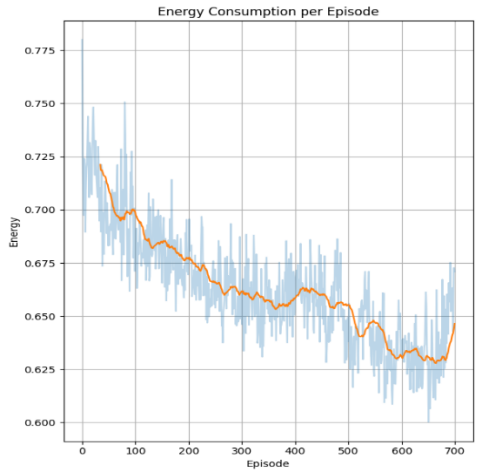

Fig. 2 Energy consumption per episode (with rolling mean).

Fig 2 shows energy per episode (mAh/episode) during training where x-axis presents the number of episodes and y-axis present the energy consumption. The rolling mean exhibits a clear downward trend: it starts around 0.72 mAh/episode in the early training phase and reaches a minimum near 0.63-0.64 mAh/episode in the later phase by roughly 580-650, before a small rebound in the final episodes (ending around ≈0.64-0.65). Overall, this corresponds to an approximate 10-12% reduction relative to the early-training level (≈0.72 → ≈0.64). This corresponds to an ideal lifetime gain factor of approximately $F \approx 1/(1-0.11)$, i.e., about +10-13% longer lifetime under comparable operating conditions.

B. Timeliness and Reliability

Fig. 3 reports packet outcomes per episode. The on-time delivery rate increases from about 56% to 66-68% over the later training region, indicating improved timeliness as the policy converges. Drop rate remains in a similar band (~20%-24%) and does not increase to "buy" energy savings. This indicates that the controller does not improve battery usage by simply sending less; it improves by choosing better

combinations of QoS and interval for the observed conditions, rather than simply reducing transmissions.

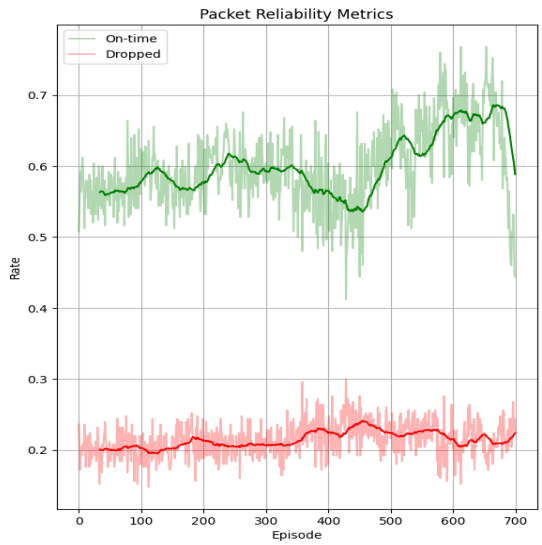

Fig. 3 Packet reliability metrics (on-time vs dropped).

C. Reward vs Energy

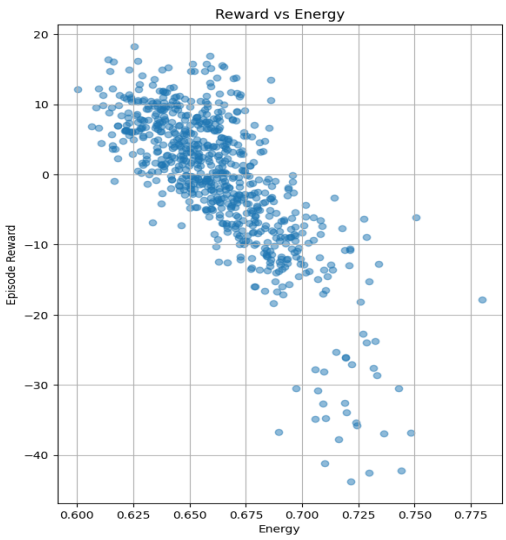

Fig. 4 Episode reward versus energy.

Fig. 4 plots episode reward versus energy. The scatter shows a clear negative association: higher rewards concentrate at lower energy (≈0.62-0.66), while high-energy episodes (≥0.70) are dominated by low returns, including several severe negative outliers.

This diagnostic indicates that the scalar reward is not "washing out" energy via cancellation; policies that spend substantially more energy are, in practice, not compensated by proportional gains in reliability/timeliness under the current weighting.

The correlation does not mean that the reward factor alone causes the change in the slope. High-energy episodes might happen at the same time as worse traffic or delays. That means that the learned policy achieves its best returns when energy use is low.

D. Qualitative Analysis of the Converged Policy

Fig. 5 summarizes the marginal action footprint over QoS and reporting interval for the final policy snapshot. The policy is strongly non-uniform and concentrates on a small set of combinations:

- QoS2 with 30 s: 0.40 (dominant)
- QoS1 with 30 s: 0.30
- QoS1 with 5 s: 0.10
- QoS2 with 5 s: 0.10
- QoS0 with 60 s: 0.10

All other QoS and interval pairs have near-zero mass. This pattern is consistent with a small number of stable "modes": a baseline 30 s operating regime (QoS1/QoS2), an escalation regime at 5 s (QoS1/QoS2), and an energy-saving regime at 60 s (QoS0).

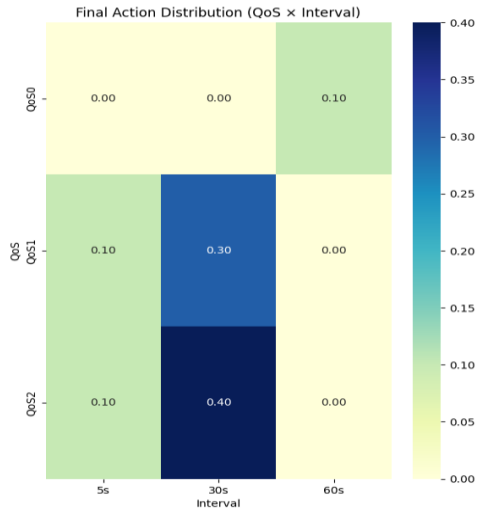

Fig. 5 Policy diagnostics.

E. Safety-Layer Test

To isolate the contribution of the safety mechanism, a test or ablation was run by removing (i) hard action masking and (ii) criticality-conditioned Q-value gating, while keeping the same DDQN architecture, state/action space, and training budget. Table 1 reports results over the last 100 episodes. With safety enabled, the episode return increases from 4.782 to 8.358, mainly because unsafe decisions that trigger large penalties are avoided. The safety layer also provides consistent KPI gains: on-time delivery improves from 63.8% to 64.8%, the drop rate decreases from 22.5% to 21.5%, average latency drops from 391.5 ms to 379.4 ms, and energy per episode decreases from 0.6498 mAh to 0.6355 mAh (≈2.2% reduction which refers only to the incremental effect of the safety layer relative to the DDQN ablation without masks and gating; it is distinct from the larger 10-12% reduction observed across the overall training trajectory). These results indicate that masking and gating act as a stabilizer that prevents clearly poor actions in risky states, while preserving the DDQN's ability to learn adaptive behaviour.

TABLE I. SAFETY-LAYER ABLATION (LAST 100 EPISODES; HIGHER ON-TIME, LOWER DROP/LATENCY/ENERGY ARE BETTER).

Setting	Return	On time (%)	Drop (%)	Avg latency (ms)	Energy (mAh/ episode)
DDQN + safety (masks + gating)	8.358	64.8	21.5	379.4	0.6355
DDQN ablation (no masks + no gating)	4.782	63.8	22.5	391.5	0.6498
Δ (safety − ablation)	3.576	+1 pp	−1 pp	−12.1	−0.0143

I. Conclusion

This article introduced a safety-constrained DDQN controller specifically designed to optimize hybrid CoAP-MQTT IoT pipelines. By synthesizing real-time context signals—including battery levels, message backlog, and network conditions—the controller dynamically adjusts MQTT QoS and reporting intervals to meet fluctuating demands. Experimental results demonstrate that the learned policy increases on-time delivery from roughly 56% to about 66-68% over training, while reducing per-episode energy from about 0.72 mAh/episode in the early-training phase to about 0.64-0.65 mAh/episode in the later phase, corresponding to an approximate 10-12% reduction relative to the early-training level. The reward-energy diagnostic further indicates that the policy improves efficiency without simply sacrificing reliability. The Dockerized CoAP-MQTT implementation also supports reproducible end-to-end evaluation.

The current study is limited to simulation, fixed reward coefficients, and a simplified discretized setting with static and heuristic baselines. Future work will consider richer multi-sensor scenarios, stronger adaptive baselines, and security-aware deployment.

References

[1] Victor Seoane, C. G.-R. (2021). Performance evaluation of CoAP and MQTT with security support for IoT environments. 197. Art. no. 108338, 2021.

[2] A. Zanella, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi, “Internet of Things for Smart Cities,” IEEE Internet of Things Journal, vol. 1, no. 1, pp. 22–32, Feb. 2014.

[3] Z. Shelby, K. Hartke, and C. Bormann, “The Constrained Application Protocol (CoAP),” Internet Engineering Task Force (IETF), RFC 7252, Jun. 2014.

[4] Andrew Banks, R. G. MQTT Version 3.1.1 Errata 01. OASIS Standard, 2014.

[5] D. Bandyopadhyay and J. Sen, “Internet of Things—Applications and Challenges in Technology and Standardization,” Wireless Personal Communications, vol. 58, no. 1, pp. 49–69, 2011.

[6] Erdal ÖZDOĞAN, Osman Ayhan ERDEM, Ahmet Nusret ÖZALP, “Adaptive Hybrid Application Protocol for IoT,” Acta Polytechnica Hungarica, vol. 21, no. 2, pp. , 2024.

[7] M.Seetharama Prasad, “Efficient power management in IoT communication protocols,” International Journal of Hybrid Information Technology, vol. 7, no. 3, 2025.

[8] C. Innamorati, A. E. C. Redondi, and M. Cesana, “Designing a broker extension for seamless CoAP and MQTT interoperability,” in Proc. 2024 IEEE 8th Forum on Research and Technologies for Society and Industry Innovation (RTSI), Milano, Italy, 2024, pp. 437–442.

[9] E. Palavras, K. Fysarakis, I. Papaefstathiou, and I. Askoxylakis, “SeMIBIoT: Secure multi-protocol integration bridge for the IoT,” in Proc. IEEE Int. Conf. Communications (ICC), Kansas City, MO, USA, 2018, pp. 1–7.

[10] H. Derhamy, Towards Interoperable Industrial Internet of Things: An On-Demand Multi-Protocol Translator Service, Licentiate thesis, Luleå Univ. of Technology, Luleå, Sweden, 2016.

[11] C.-H. Lee, Y.-W. Chang, C.-C. Chuang, and Y.-H. Lai, “Interoperability enhancement for Internet of Things protocols based on software-defined network,” in Proc. IEEE 5th Global Conf. Consumer Electronics (GCCE), 2016, pp.1-2.

[12] V. Mnih et al., “Human-level control through deep reinforcement learning,” Nature, vol. 518, no. 7540, pp. 529–533, 2015.

[13] O. Bouhamed and O. O. Bouachir, M. Aloqaily and I. A. Ridhawi, “Lightweight IDS for UAV networks: A periodic deep reinforcement learning-based approach,” in Proc. IFIP/IEEE Int. Symp. Integrated Network Management (IM), 2021, pp. 1032-1037.

[14] Donta, P.K., Srirama, S.N., Amgoth, T. et al. “iCoCoA: Intelligent congestion control algorithm for CoAP using deep reinforcement learning,” Journal of Ambient Intelligence and Humanized Computing (Springer), vol. 14, pp. 2951–2966, 2023.

[15] E. González, S. Camacho-Leon, R. Pena et al., “Comparative analysis of power consumption between MQTT and HTTP protocols in an IoT platform for remote real-time monitoring of long-term cold chain transport operations,” Sensors, vol. 23, no. 10, Art. no. 4896, 2023.

[16] D. Thangavel, X. Ma, A. Valera, H.-X. Tan, and C. K. Y. Tan, “Performance evaluation of MQTT and CoAP via a common middleware,” in Proc. IEEE 9th Int. Conf. Intelligent Sensors, Sensor Networks and Information Processing (ISSNIP), 2014, pp. 1–6.

[17] H. van Seijen et al., “Hybrid reward architecture for reinforcement learning,” arXiv:1706.04208, 2017.

[18] K. Kavitha and G. Suseendran, "Priority based Adaptive Scheduling Algorithm for IoT Sensor Systems," 2019 International Conference on Automation, Computational and Technology Management (ICACTM), London, UK, 2019, pp. 361-366.

[19] G. Dulac-Arnold et al., “Challenges of real-world reinforcement learning,” Nature Machine Intelligence, vol. 3, no. 6, pp. 467–475, 2021.

[20] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized Experience Replay," in Proc. Int. Conf. Learn. Represent. (ICLR), 2016.

[21] T. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), 2017, pp. 2980–2988.

[22] M. Jaderberg et al., “Reinforcement learning with unsupervised auxiliary tasks,” in Proc. Int. Conf. Learning Representations (ICLR), 2017.

[23] R. Meyes, M. Terhorst, C. R. G. Dreischer, and T. Meisen, “Ablation studies in artificial neural networks,” arXiv:1901.08644, 2019.